



PureTime: Noise-Free Serverless Execution Time Measurement System

1. 요약 (Executive Summary)

[핵심 기여](#)

[활용 분야](#)

2. 문제 정의 및 배경

[2.1 서버리스 컴퓨팅의 성능 변동성 문제](#)

[2.2 Noisy Neighbor 현상](#)

[2.2.1 CPU 경합](#)

[2.2.2 Network 경합](#)

[2.2.3 Block I/O 경합](#)

[2.2.4 Softirq 간섭](#)

[2.3 기존 접근 방식의 한계](#)

[2.4 본 연구의 차별점](#)

3. 시스템 설계

[3.1 전체 아키텍처](#)

[3.1.1 eBPF Tracer \(커널 공간\)](#)

[3.1.2 Loader \(유저 공간, C\)](#)

[3.1.3 Analyzer \(유저 공간, Python\)](#)

[3.2 데이터 흐름](#)

[3.3 기술 선택 근거: 왜 eBPF인가?](#)

4. 트레이싱 전략

[4.1 설계 원칙: 의미있는 지연만 측정](#)

[4.2 CPU 스케줄링 트레이싱](#)

[4.3 Network TX \(송신\) 트레이싱](#)

[4.4 Block I/O 트레이싱](#)

[4.5 Softirq 트레이싱](#)

5. Wait 계산 알고리즘

[5.1 핵심 개념](#)

[5.2 CPU Wait 계산](#)

[5.3 Network Wait 계산](#)

[5.4 Block I/O Wait 계산](#)

[5.5 Softirq 오버헤드 분류](#)

6. Noise-Free Makespan 계산

[6.1 서버리스 함수의 실행 특성](#)

[6.2 Wait 구간의 Overlap 문제](#)

[6.3 Interval Merge 알고리즘](#)

[6.4 최종 Makespan 계산](#)

7. 출력 메트릭 및 결과 해석

[7.1 타임라인 슬롯 분류](#)

[7.1.1 실행 시간에 포함되는 슬롯](#)

[7.1.2 Noisy Neighbor로 인한 Wait 슬롯 \(제거 대상\)](#)

[7.2 주요 출력 메트릭](#)

[7.2.1 핵심 메트릭](#)

[7.2.2 Breakdown 메트릭](#)

[7.3 결과 해석 가이드](#)

[wait_percentage가 높은 경우 \(30% 이상\)](#)

[특정 wait 유형이 지배적인 경우](#)

8. 환경 요구사항 및 구현 고려사항

[8.1 운영 체제 요구사항](#)

[8.2 서버리스 플랫폼 호환성](#)

[8.3 NIC Offload 비활성화](#)

8.4 I/O 스케줄러 설정 (선택)	
8.5 권한 요구사항	
9. 제약사항 및 한계	
9.1 소켓 없는 패킷	
9.2 Softirq 귀속의 한계	
9.3 타임라인 재구성의 보수성	
9.4 측정 오버헤드	
9.5 메모리/디스크 사용	
10. 관련 연구 및 차별점	
10.1 서버리스 성능 분석 연구	
10.2 성능 간섭 연구	
10.3 eBPF 기반 트레이싱 연구	
10.4 PureTime의 차별화된 기여	
11. 향후 연구 방향	
11.1 Network RX 트레이싱	
11.2 LLC Cache Contention	
11.3 이기종 하드웨어 정규화	
11.4 실시간 대시보드	
12. 결론	
12.1 핵심 기여 요약	
12.2 실용적 의의	
12.3 향후 전망	
성능 평가를 위한 실험	
Puretime의 노이즈 제거 정확도 분석	
Baseline과의 정확도 분석	
노이즈 유형별 정확도 분석	
노이즈 강도별 정확도 분석	
시스템 오버헤드 측정	
실행 시간 지연 측정	
자원 소비량 측정	
Case Study	
False Alarm 잡는 카나리 배포	
per Resource Contention Aware Scheduling	

1. 요약 (Executive Summary)

서버리스 컴퓨팅 환경에서 함수 실행 시간은 동일한 코드와 입력에도 불구하고 높은 변동성을 보인다. Azure Functions 데이터셋 분석 결과 392개 함수 중 86.79%가 CV(Coefficient of Variation) 10% 이상을 나타냈으며, Huawei Private Cloud 데이터셋에서는 200개 함수 중 99%가 유사한 수준의 변동성을 보였다. 이러한 변동성의 주된 원인은 **Noisy Neighbor 현상**—즉, 동일 물리 서버에서 실행되는 다른 테넌트의 워크로드로 인한 자원 경쟁이다.

PureTime은 Linux eBPF(extended Berkeley Packet Filter) 기술을 활용하여 커널 레벨에서 자원 경쟁으로 인한 대기 시간을 정밀하게 측정하고, 이를 원본 실행 시간에서 제거하여 **Noise-Free Makespan**—즉, noisy neighbor가 없었을 때의 순수 함수 실행 시간—을 계산하는 시스템이다.

핵심 기여

- 정밀한 노이즈 측정:** CPU 스케줄링, 네트워크 송신, 블록 I/O, softirq 네 가지 경쟁 지점에서 noisy neighbor로 인한 대기 시간을 커널 레벨에서 측정
- Noise-Free Makespan 계산:** 측정된 노이즈 구간의 overlap을 고려한 interval merge 알고리즘을 통해 정확한 순수 실행 시간 산출
- 컨테이너 수준 귀속:** cgroup ID 기반으로 각 이벤트를 함수 인스턴스(컨테이너)에 정확히 귀속

활용 분야

- 리소스 최적화:** 노이즈 제거된 실행 시간 기반의 정확한 자원 할당 결정
- 스케줄러 연구:** Fluctuation-aware 스케줄러 개발 및 평가를 위한 기준 데이터 제공
- 성능 벤치마킹:** 환경 노이즈를 제거한 공정한 함수 성능 비교

- 카나리 배포: 성능 회귀 탐지 시 false positive/negative 감소

2. 문제 정의 및 배경

2.1 서버리스 컴퓨팅의 성능 변동성 문제

서버리스 컴퓨팅(Function-as-a-Service)은 클라우드 제공자가 인프라 관리를 담당하고, 사용자는 함수 코드만 제공하는 모델이다. 비용 효율성을 위해 클라우드 제공자는 여러 테넌트의 함수를 동일한 물리 서버에서 실행하는 멀티테넌시(multi-tenancy) 아키텍처를 채택한다.

이러한 아키텍처에서 개별 함수의 실행 시간은 외부 요인에 의해 크게 영향받는다. 동일한 함수를 동일한 입력으로 100회 실행하더라도 실행 시간이 수십 퍼센트 이상 변동하는 것이 일반적이다.

[그림 2.1: 서버리스 함수 실행 시간 분포 예시]
 - X축: 실행 횟수
 - Y축: 실행 시간 (ms)
 - 동일 함수의 반복 실행에서 나타나는 높은 분산을 시각화

2.2 Noisy Neighbor 현상

Noisy Neighbor란 동일한 물리적 자원을 공유하는 다른 테넌트의 워크로드가 내 워크로드의 성능에 부정적 영향을 미치는 현상이다. 서버리스 환경에서 이 현상은 다음 네 가지 주요 경합 지점에서 발생한다.

2.2.1 CPU 경합

운영체제의 스케줄러는 실행 가능한 모든 프로세스를 CPU의 run queue에서 관리한다. 내 프로세스가 실행 가능 상태(TASK_RUNNING)가 되어 run queue에 들어가더라도, 다른 테넌트의 프로세스가 먼저 CPU를 할당받으면 내 프로세스는 대기해야 한다.

시간 →
 [다른 테넌트 실행] [다른 테넌트 실행] [내 프로세스 실행]
 ↑
 내 프로세스 enqueue
 |← CPU Wait →|

2.2.2 Network 경합

네트워크 패킷이 전송될 때, 커널의 Qdisc(Queue Discipline)를 거친다. 내 패킷이 Qdisc에 들어갔을 때 다른 테넌트의 패킷이 이미 큐에 있으면, 그 패킷들이 먼저 전송된 후에야 내 패킷이 NIC로 전달된다.

Qdisc Queue:
 [패킷A-타테넌트] [패킷B-타테넌트] [내 패킷]
 ↓ ↓
 먼저 전송 대기 후 전송

2.2.3 Block I/O 경합

디스크 I/O 요청은 블록 레이어의 I/O 스케줄러 큐에서 관리된다. 내 I/O 요청이 큐에 들어갔을 때 다른 테넌트의 요청이 앞에 있거나 디바이스가 다른 요청을 처리 중이면, 내 요청은 추가로 대기해야 한다.

2.2.4 Softirq 간섭

Linux에서 네트워크 패킷 수신, 블록 I/O 완료 등의 처리는 softirq 컨텍스트에서 이루어진다. Softirq는 현재 실행 중인 프로세스의 컨텍스트를 "빌려서" 실행되므로, 다른 테넌트의 I/O를 처리하는 softirq가 발생하면 내 프로세스의 실행이 일시 중단된다.

2.3 기존 접근 방식의 한계

이러한 경합으로 인해 동일한 함수를 동일한 입력으로 실행해도 실행 시간이 크게 달라진다. 문제는 현재의 메트릭(단순 실행 시간)으로는 이 변동성이 함수 코드 자체의 문제인지, 환경에서 오는 노이즈인지 구분할 수 없다는 것이다. 전통적으로 성능 측정의 변동성 문제는 다음과 같은 방법으로 대응해왔다:

방법	접근 방식	한계
반복 측정	수십~수백 회 실행 후 평균/중앙값 사용	시간과 비용 소모, 근본적 해결 아님
백분위수 사용	P90, P95 등 사용	노이즈 원인 파악 불가, 이상치 포함 가능
격리 환경	전용 서버에서 측정	실제 운영 환경과 괴리, 비용 증가
통계적 필터링	Outlier 제거	임의적 기준, 실제 노이즈와 무관할 수 있음

하지만 이러한 방법들은 **노이즈의 존재를 가정하고 완화**하는 것이지, **노이즈의 정체를 파악하고 정밀하게 제거**하는 것이 아니다.

2.4 본 연구의 차별점

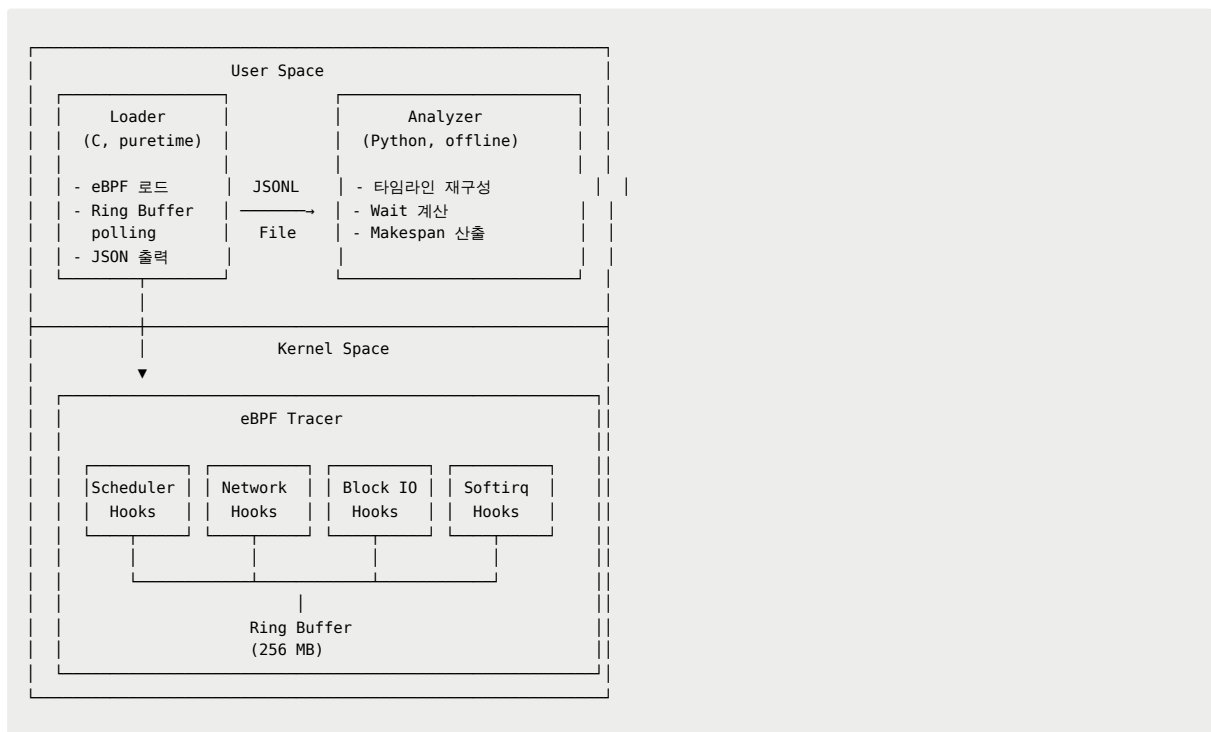
PureTime은 **노이즈를 직접 측정**하는 근본적으로 다른 접근을 취한다. eBPF를 활용하여 커널 레벨에서 자원 경쟁으로 인한 대기 시간을 정밀하게 측정한다. 이때 단순히 "대기가 발생했다"가 아니라 "다른 테넌트 때문에 대기가 발생했다"를 식별하고, 다른 테넌트로 인해 발생한 대기 시간을 제거하여 Noisy neighbor가 없었을 때의 순수 함수 실행 시간(Noise-Free Makespan)을 계산하는 시스템을 구현한다.

이를 통해 단일 실행에서도 노이즈가 제거된 순수 실행 시간을 얻을 수 있어, 적은 샘플로도 신뢰성 높은 성능 비교가 가능하다. 또한 함수 실행 시간을 보다 자세히 분석하여 함수가 느려졌을 때 그 원인이 코드 변경 때문인지, 환경의 노이즈 때문인지 구분할 수 있다. 이를 통해 불필요한 디버깅 시간을 줄이고, 실제 문제에 집중할 수 있다. 시스템 관리자의 입장에서는 CPU, Network, Block I/O 중 어디서 얼마나 대기했는지 breakdown을 제공하여, 최적화 포인트를 명확하게 파악할 수 있다.

3. 시스템 설계

3.1 전체 아키텍처

PureTime은 세 개의 주요 컴포넌트로 구성된다:



3.1.1 eBPF Tracer (커널 공간)

커널의 주요 지점에 hook을 설치하여 스케줄링, 네트워크 전송, 블록 I/O, softirq 관련 이벤트를 캡처한다. 캡처된 이벤트는 Ring Buffer를 통해 유저 공간으로 전달된다.

주요 특징:

- CO-RE(Compile Once - Run Everywhere) 지원으로 커널 버전 간 호환성 확보
- BTF(BPF Type Format) 활용으로 커널 구조체에 안전하게 접근

- cgroup ID 기반 필터링으로 컨테이너화된 프로세스만 추적

3.1.2 Loader (유저 공간, C)

eBPF 프로그램을 커널에 로드하고 attach하는 역할을 담당한다. Ring Buffer에서 이벤트를 polling하여 JSON Lines 형식으로 로그 파일에 저장한다.

주요 기능:

- eBPF 프로그램 lifecycle 관리 (open → load → attach → detach)
- Ring Buffer 이벤트 수신 및 JSON 변환
- 버퍼링된 I/O로 오버헤드 최소화 (4KB 버퍼)
- 시그널 핸들링을 통한 graceful shutdown

3.1.3 Analyzer (유저 공간, Python)

저장된 로그 파일을 파싱하여 cgroup별로 타임라인을 재구성하고, noisy neighbor로 인한 대기 시간을 계산한 후 noise-free makespan을 산출한다.

분석 단계:

1. JSONL 파싱 및 이벤트 분류
2. cgroup별 이벤트 그룹화
3. 이벤트 쌍 매칭 (enqueue ↔ switch, queue ↔ xmit 등)
4. Wait 구간 식별 및 noisy neighbor 귀속
5. Interval merge를 통한 overlap 제거
6. 최종 noise-free makespan 계산

3.2 데이터 흐름



서버리스 함수가 실행되면 다양한 커널 이벤트가 발생한다. 예를 들어 프로세스가 깨어나면 `sched_wakeup` 이벤트가, CPU를 할당받으면 `sched_switch` 이벤트가, 패킷을 전송하면 `net_dev_queue` → `net_dev_start_xmit` → `net_dev_xmit` 이벤트가 순차적으로 발생한다.

eBPF Tracer는 이러한 이벤트마다 타임스탬프, cgroup ID, CPU 번호, 이벤트 유형별 메타데이터(패킷 주소, I/O 요청 주소, 프로세스 정보 등)를 기록한다. 특히 cgroup ID는 Knative 환경에서 각 함수 인스턴스(컨테이너)를 고유하게 식별하는 데 사용된다.

Analyzer는 수집된 이벤트를 cgroup별로 그룹화하고, 이벤트 쌍(예: run queue 진입 → switch-in, queue → dequeue)을 매칭하여 각 구간의 소요 시간을 계산한다. 그리고 그 구간 동안 다른 cgroup의 요청이 처리되었는지 확인하여 noisy neighbor로 인한 대기 시간을 식별한다.

3.3 기술 선택 근거: 왜 eBPF인가?

대안 기술	한계	eBPF의 장점
커널 모듈	커널 버전 종속, 보안 위험, 재부팅 필요	안전한 샌드박스, 동적 로드/언로드
SystemTap	복잡한 설정, 오버헤드 큼	저오버헤드, 프로덕션 사용 가능
ftrace	유연성 부족, 복잡한 필터링 어려움	프로그래밍 가능한 필터링
perf	샘플링 기반, 전수 조사 어려움	모든 이벤트 캡처 가능
애플리케이션 계측	커널 레벨 정보 접근 불가	커널 내부 상태 직접 접근

eBPF는 커널 내에서 안전하게 실행되는 샌드박스 프로그램으로, 다음과 같은 이점을 제공한다:

1. 낮은 오버헤드: JIT 컴파일로 네이티브에 가까운 성능
2. 안전성: Verifier가 메모리 접근, 무한 루프 등을 사전 검증
3. 유연성: 복잡한 필터링 로직을 프로그래밍 가능
4. 포터빌리티: CO-RE로 다양한 커널 버전 지원
5. 프로덕션 준비: 시스템 재시작 없이 동적 활성화/비활성화

4. 트레이싱 전략

4.1 설계 원칙: 의미있는 지연만 측정

모든 커널 이벤트를 트레이싱하면 오버헤드가 너무 크다. PureTime은 아래 기준을 통해 **noisy neighbor에 의해 의미있게 영향받는 구간만** 선별하여 트레이싱한다.

1. 해당 구간에서 **대기가 발생**한다
2. 그 대기의 원인이 **다른 테넌트의 요청이 먼저 처리되고 있기** 때문이다
3. 다른 테넌트가 없었다면 그 대기가 **발생하지 않았거나 더 짧았을** 것이다

4.2 CPU 스케줄링 트레이싱

측정 구간: 프로세스가 실행 가능 상태가 된 시점(Run queue 진입)부터 실제로 CPU를 할당받아 실행(Running)되는 시점까지의 **run queue 대기 시간**.

Hook Points:

Hook	트리거 조건	수집 정보
<code>fentry/enqueue_task</code>	프로세스가 run queue에 들어갈 때	타임스탬프, cgroup ID, PID, CPU
<code>tp_btf/sched_switch</code>	컨텍스트 스위칭 발생 시	prev/next 프로세스 정보, 상태

특별 처리: prev 프로세스가 switch-out될 때 `TASK_RUNNING` 상태이면, sleep 없이 run queue로 돌아가는 것이므로 `enqueue_task`를 거치지 않지만 대기 구간으로 처리해야 한다.

수집 데이터 구조:

```
struct sched_event {
    struct event_header hdr;           // 공통 헤더
    __u32 prev_pid, next_pid;         // 이전/다음 프로세스 PID
    __u64 prev_cgroup_id;             // 이전 프로세스 cgroup
    __u64 next_cgroup_id;             // 다음 프로세스 cgroup
    char prev_comm[16];               // 이전 프로세스 이름
    char next_comm[16];               // 다음 프로세스 이름
};
```

```

__s64 prev_state; // 이전 프로세스 상태
};

```

내 프로세스가 run queue에 들어갔는데 다른 테넌트의 프로세스가 먼저 스케줄되면, 그 다른 프로세스 때문에 내가 못나가고 대기해야 한다. > 즉, 다른 테넌트의 프로세스가 없었으면 내 테넌트의 프로세스가 그 시점에 나갔을 것이다. > 다른 테넌트가 나간 시점부터 실제 내 테넌트가 나간 시점까지가 Noisy Neighbor로 인해 발생한 추가 지연이다.

4.3 Network TX (송신) 트레이싱

측정 구간: 패킷이 Qdisc(Queue Discipline)에 들어간 시점부터 실제로 NIC로 전송되는 시점까지의 **대기 시간**.

Hook Points:

Hook	트리거 조건	의미
<code>tp_btf/net_dev_queue</code>	패킷이 Qdisc에 enqueue	Qdisc 진입 시점
<code>tp_btf/net_dev_start_xmit</code>	패킷이 Qdisc에서 dequeue	드라이버 전달 시점
<code>tp_btf/net_dev_xmit</code>	NIC로 전송 완료	실제 전송 완료 시점

중요 전제조건: NIC의 offload 기능을 반드시 비활성화해야 한다.

Offload 기능	문제점
TSO (TCP Segmentation Offload)	여러 패킷이 하나로 합쳐져 커널을 통과
GSO (Generic Segmentation Offload)	커널에서 보는 패킷과 실제 wire 패킷이 불일치
GRO (Generic Receive Offload)	수신 측에서 패킷 합침

cgroup 추적의 특수성: 네트워크 이벤트 컨텍스트에서는 `bpf_get_current_cgroup_id()` 가 항상 정확하지 않다. 따라서 TCP 연결 시점에 socket과 cgroup의 매핑을 저장하는 별도 BPF 맵을 유지한다.

```

// Socket → cgroup 매핑 저장
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 65536);
    __type(key, __u64); // socket 주소
    __type(value, __u64); // cgroup ID
} socket_cgroup_map SEC(".maps");

```

내 패킷이 Qdisc에 들어갔을 때 다른 테넌트의 패킷이 이미 큐에 있으면, 그 패킷들 때문에 내 패킷은 대기해야 한다. > 다른 테넌트의 패킷이 없었으면, 내 패킷이 그 시점에 나가서 처리되었을 것이다. > 다른 테넌트의 패킷이 나간 시점부터 내 패킷이 나간 시점까지가 Noisy Neighbor로 인해 발생한 추가 지연이다.

4.4 Block I/O 트레이싱

측정 구간: I/O 요청이 block layer에 제출된 시점부터 실제로 디바이스에서 처리 완료되는 시점까지. 이 중 **I/O 스케줄러 큐에서 대기하는 시간**이 noisy neighbor 영향을 받는다.

Hook Points:

Hook	트리거 조건	의미
<code>tp_btf/block_rq_insert</code>	I/O 요청이 스케줄러 큐에 삽입	큐 진입 시점
<code>tp_btf/block_rq_issue</code>	I/O 요청이 디바이스로 발행	큐에서 나가는 시점
<code>tp_btf/block_rq_complete</code>	I/O 처리 완료	완료 시점

수집 데이터 구조:

```

struct block_event {
    struct event_header hdr;
    __u64 request; // request 구조체 주소 (추적용)
    __u32 dev; // 디바이스 번호
    __u64 sector; // 시작 섹터
};

```

```
__u32 nr_sector;          // 섹터 수
__u32 rwflag;             // Read/Write 플래그
};
```

내 I/O 요청이 큐에 들어갔을 때 다른 테넌트의 요청이 큐에서 대기 중이면, 그 요청들이 먼저 큐에서 나가고 내 요청이 처리된다. > 다른 테넌트의 요청이 없었으면, 내 요청이 그 시점에 나가서 처리되었을 것이다. > 다른 테넌트의 요청이 나간 시점부터 내 요청이 나간 시점까지가 Noisy Neighbor로 인해 발생한 추가 지연이다.

4.5 Softirq 트레이싱

배경: Linux에서 네트워크 패킷 수신, Block I/O 완료 등의 처리는 softirq 컨텍스트에서 이루어진다. Softirq는 하드웨어 인터럽트 처리 후 또는 시스템 콜 반환 시점에 실행되며, **현재 실행 중인 프로세스의 컨텍스트를 "빌려서" 실행**된다.

Hook Points:

Hook	수집 정보
tp_btf/softirq_entry	softirq 벡터 번호, 시작 시점, 실행 중이던 프로세스 cgroup
tp_btf/softirq_exit	종료 시점

필터링: 모든 softirq를 트레이싱하면 오버헤드가 크므로, noisy neighbor와 관련된 것만 선별한다.

vec_nr	이름	설명	트레이싱 여부
1	NET_TX_SOFTIRQ	네트워크 송신	O
2	NET_RX_SOFTIRQ	네트워크 수신	O
4	BLOCK_SOFTIRQ	Block I/O 완료	O
0, 7	TIMER, SCHED	타이머, 스케줄러	X (빈도 높음)

다른 테넌트의 패킷 수신이나 I/O 완료를 처리하는 softirq가 내 프로세스 실행 중에 발생하면, 내 프로세스는 그 softirq가 완료될 때까지 실제 작업을 수행하지 못한다. 동시에 실행되는 테넌트가 많아질수록 전체 시스템의 인터럽트 빈도가 증가하므로, 이 영향도 커진다.

5. Wait 계산 알고리즘

5.1 핵심 개념

단순히 "큐에서 대기한 시간"을 모두 noisy neighbor 탓으로 돌릴 수는 없다. 예를 들어 내 요청이 큐에 혼자 있어도 디바이스 처리 시간만큼은 대기해야 한다. 핵심은 "다른 테넌트가 없었다면 발생하지 않았을 대기 시간"을 식별하는 것이다. 이를 위해 각 리소스 유형별로 **"내가 큐에 들어갔을 때 이미 다른 테넌트의 요청이 앞에 있었는지"**를 확인하고, 그로 인해 추가로 대기한 시간을 계산한다.

5.2 CPU Wait 계산

알고리즘: 내 프로세스가 run queue에 enqueue된 후 switch-in되기까지의 구간에서, **같은 CPU에서 다른 cgroup의 프로세스가 먼저 switch-in된 경우**를 찾는다.

```
t=0: 내 프로세스 enqueue (run queue 진입)
t=5: 다른 tenant의 프로세스가 switch-in (CPU 획득)
t=15: 내 프로세스가 switch-in (CPU 획득)

Wait = t=5 ~ t=15 = 10 단위시간
```

근거: 다른 테넌트의 프로세스가 없었다면 스케줄러가 내 프로세스를 더 일찍 선택했을 것이다. 다른 프로세스가 switch-in된 시점(t=5)이 바로 "내가 스케줄될 수 있었던 시점"이고, 그로부터 실제로 스케줄된 시점(t=15)까지가 다른 테넌트 때문에 추가로 대기한 시간이다.

의사 코드:

```
def calculate_cpu_wait(my_enqueue_time, my_switch_in_time, all_switch_events, my_cgroup):
```



```

wait_intervals = []

for event in all_switch_events:
    if event.cpu == my_cpu and event.cgroup != my_cgroup:
        if my_enqueue_time < event.time < my_switch_in_time:
            # 다른 테넌트가 나보다 먼저 스케줄됨
            wait_start = event.time
            wait_end = find_next_switch_out(event.cgroup) or my_switch_in_time
            wait_intervals.append((wait_start, min(wait_end, my_switch_in_time)))

return merge_intervals(wait_intervals)

```

5.3 Network Wait 계산

알고리즘: 내 패킷이 Qdisc에 들어간(`net_dev_queue`) 이후에 다른 tenant의 패킷이 dequeue되면(`net_dev_start_xmit`), 그 때부터 내 패킷이 dequeue되는 시점까지가 wait이다.

```

t=5:   내 패킷 B가 queue에 들어감
t=10:  다른 tenant 패킷 A가 dequeue됨
t=12:  내 패킷 B가 dequeue됨

Wait = t=10 ~ t=12 = 2 단위시간
(내가 나갈 수 있었는데, 다른 패킷 때문에 대기한 시간)

```

판단 기준: 다른 패킷이 내 패킷 enqueue 후에 dequeue된 경우, 그 패킷은 내 패킷보다 앞에 있었던 것이다.

5.4 Block I/O Wait 계산

알고리즘: 내 I/O 요청이 큐에 insert된 이후 다른 테넌트의 I/O 요청이 내 요청보다 먼저 큐에서 issue된 경우, 다른 요청이 issue된 시점부터 내 요청이 issue된 시점까지가 추가 대기 시간이다.

```

t=5:   내 요청 B가 insert됨 (큐에 들어감)
t=10:  다른 tenant 요청 A가 issue됨 (디바이스에서 처리 시작)
t=11:  내 요청 B가 issue됨

Wait = t=10 ~ t=11 = 1 단위시간
(요청 A로 인해 큐에서 뺄리지 못하고 추가로 기다린 시간)

```

5.5 Softirq 오버헤드 분류

Softirq 구간은 무조건 noisy neighbor로 분류할 수 없다. 그 softirq가 **내 I/O를 처리하는 것**일 수도 있기 때문이다.

분류 방법: Softirq entry/exit 구간 동안 발생한 네트워크/블록 I/O 이벤트의 cgroup을 확인한다. 내 cgroup의 I/O를 처리한 비율만큼은 "정당한" 오버헤드이고, 다른 cgroup의 I/O를 처리한 비율만큼은 noisy neighbor로 인한 오버헤드이다.

```

Softirq 구간: 100ns
구간 내 I/O 이벤트:
- 내 cgroup 패킷 2개 처리
- 다른 cgroup 패킷 3개 처리

→ 내 몫: 100ns × (2/5) = 40ns (실행 시간에 포함)
→ 남의 몫: 100ns × (3/5) = 60ns (noisy neighbor wait로 분류)

```

6. Noise-Free Makespan 계산

6.1 서버리스 함수의 실행 특성

서버리스 함수는 일반적으로 **직렬 실행 패턴**을 따른다. 함수 내부의 애플리케이션 로직은 병렬화되지 않으며, 다음과 같은 순서로 실행된다:

APP 로직 실행 → I/O 요청 → I/O 완료 대기 → APP 로직 실행 → I/O 요청 → ...

즉, 애플리케이션 코드가 I/O를 요청하면 그 결과가 올 때까지 다른 애플리케이션 로직을 실행하지 않고 대기한다. 이는 vCPU가 여러 개 할당되어 있어도 마찬가지이다.

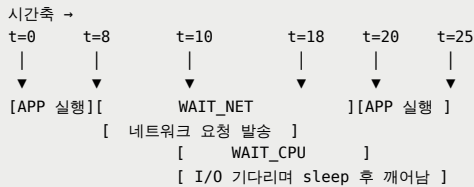
단, I/O 자체는 별도 하드웨어(NIC, SSD)에서 처리되므로, 애플리케이션이 대기하는 동안에도 I/O 처리는 진행될 수 있다.

6.2 Wait 구간의 Overlap 문제

여러 종류의 wait가 시간적으로 겹칠 수 있다. 예를 들어 애플리케이션이 I/O 결과를 기다리면서 sleep 상태에 있다가 깨어났는데, 마침 CPU를 다른 테넌트가 점유하고 있어서 run queue에서도 대기해야 하는 상황이다.

```
t=0~10: APP 실행, t=8에 네트워크 요청 발송
t=8~18: WAIT_NET (내 패킷이 Qdisc에서 대기, 다른 패킷 뒤에)
t=10~20: WAIT_CPU (APP이 I/O 결과 기다리며 sleep했다가 깨어났는데 run queue 대기)
t=18: 네트워크 전송 완료
t=20~25: APP 실행 재개
```

개별 합산: $WAIT_CPU(10) + WAIT_NET(10) = 20$
실제 overlap 구간: $t=10\sim18$ (8 단위시간이 중복 계산됨)



- 개별 합산: $WAIT_CPU(10) + WAIT_NET(10) = 20$
- 실제 overlap 구간: $t=10\sim18$ (8 단위시간이 중복 계산됨)

단순히 모든 wait를 합산하면 $t=10\sim18$ 구간이 **이중으로 계산**된다. 실제로는 그 시간 동안 두 가지 대기가 "동시에" 발생한 것이므로, **한 번만 빼야** 한다.

6.3 Interval Merge 알고리즘

모든 wait 구간을 시간 interval로 표현한 후, 겹치는 구간을 병합하여 "순수 wait 시간"을 계산한다.

알고리즘:

```
def merge_intervals(intervals):
    """겹치는 구간을 병합"""
    if not intervals:
        return []

    # 1. 시작시각 기준으로 정렬
    sorted_intervals = sorted(intervals, key=lambda x: x[0])

    merged = [sorted_intervals[0]]

    for current in sorted_intervals[1:]:
        last = merged[-1]

        # 2. 겹치거나 연속되면 병합
        if current[0] <= last[1]:
            merged[-1] = (last[0], max(last[1], current[1]))
        else:
            merged.append(current)
```

```
return merged

def calculate_unique_wait(all_wait_intervals):
    merged = merge_intervals(all_wait_intervals)
    return sum(end - start for start, end in merged)
```

예시:

```
입력: [(0,10), (5,15), (20,30)]
정렬 후: [(0,10), (5,15), (20,30)]
병합 과정:
- (0,10) 시작
- (5,15): 5 < 10이므로 병합 → (0,15)
- (20,30): 20 > 15이므로 새 구간
병합 후: [(0,15), (20,30)]
순수 wait 시간: 15 + 10 = 25 (단순 합산 30에서 overlap 5 제거)
```

6.4 최종 Makespan 계산

```
noise_free_makespan = original_makespan - unique_wait_time
```

여기서:

- `original_makespan`: 함수 실행의 원본 소요 시간 (첫 이벤트부터 마지막 이벤트까지의 wall-clock 시간)
- `unique_wait_time`: Overlap이 제거된 순수 noisy neighbor 대기 시간

7. 출력 메트릭 및 결과 해석

7.1 타임라인 슬롯 분류

분석 결과로 생성되는 타임라인의 각 구간은 다음 유형으로 분류된다.

7.1.1 실행 시간에 포함되는 슬롯

슬롯 유형	설명
APP_LOGIC	애플리케이션 코드가 CPU에서 실제로 실행된 시간. 함수의 본질적인 연산 시간
NETWORK_PROCESSING	내 패킷이 실제로 NIC에서 처리되는 시간 (Tx ring → wire)
BLOCK_IO_PROCESSING	내 I/O 요청이 디바이스에서 처리되는 시간 (issue → complete)
SOFTIRQ_SELF	내 I/O를 처리하는 softirq 시간

7.1.2 Noisy Neighbor로 인한 Wait 슬롯 (제거 대상)

슬롯 유형	설명
WAIT_CPU	다른 테넌트가 CPU를 점유해서 run queue에서 대기한 시간
WAIT_NET	다른 테넌트의 패킷이 Qdisc에서 먼저 처리되기를 기다린 시간
WAIT_BIO	다른 테넌트의 I/O 요청이 완료되기를 기다린 시간
SOFTIRQ_OTHER	다른 테넌트의 I/O를 처리하는 softirq로 인해 중단된 시간

7.2 주요 출력 메트릭

7.2.1 핵심 메트릭

메트릭	설명	계산식
<code>original_makespan</code>	원본 실행 시간	마지막 이벤트 - 첫 이벤트

메트릭	설명	계산식
<code>noise_free_makespan</code>	노이즈 제거 실행 시간	$\text{original} - \text{unique_wait}$
<code>total_unique_wait</code>	순수 대기 시간 (overlap 제거)	merge후 총 길이
<code>wait_percentage</code>	대기 시간 비율	$(\text{unique_wait} / \text{original}) \times 100$

7.2.2 Breakdown 메트릭

각 유형별 대기 시간을 개별적으로 제공한다. Overlap 때문에 이 값들의 합이 `total\_unique\_wait` 보다 클 수 있다.

메트릭	설명
<code>wait_cpu</code>	CPU run queue 대기 시간 합계
<code>wait_net</code>	Network Qdisc 대기 시간 합계
<code>wait_bio</code>	Block I/O 스케줄러 대기 시간 합계
<code>softirq_other</code>	다른 테넌트 I/O 처리 softirq 오버헤드 합계
<code>softirq_self</code>	내 I/O 처리 softirq 시간 합계 (참고용)

7.3 결과 해석 가이드

wait_percentage가 높은 경우 (30% 이상)

의미: 해당 함수가 noisy neighbor의 영향을 많이 받고 있다.

해석: 원본 실행 시간만 보면 함수가 느려 보이지만, 실제로는 환경 때문이다. 서버의 부하가 높은 시간대이거나, 동일 노드에 리소스를 많이 사용하는 다른 함수가 있을 수 있다.

조치:

- 부하가 낮은 시간대에 재측정
- 노드 배치 정책 검토
- 자원 격리 강화 고려

특정 wait 유형이 지배적인 경우

지배적 유형	의미	조치
<code>wait_cpu</code>	CPU 경합 심함	CPU 자원 증가 또는 노드 분산
<code>wait_net</code>	네트워크 경합 심함	네트워크 대역폭 확인, QoS 정책 검토
<code>wait_bio</code>	스토리지 경합 심함	디스크 I/O 스케줄러 조정, SSD 업그레이드

8. 환경 요구사항 및 구현 고려사항

8.1 운영 체제 요구사항

항목	요구사항	비고
배포판	Ubuntu 24.04 LTS	테스트 완료
커널	6.8.0-generic 이상	BTF, CO-RE 지원 필수
BTF 지원	필수	<code>/sys/kernel/btf/vmlinux</code> 존재 확인

BTF(BPF Type Format)는 커널 타입 정보를 BPF 프로그램에 노출하여 CO-RE(Compile Once - Run Everywhere)를 가능하게 한다. 대부분의 최신 배포판은 BTF가 활성화된 커널을 제공한다.

8.2 서버리스 플랫폼 호환성

기본 대상: Knative 환경

호환 조건: 각 함수 인스턴스가 독립된 컨테이너로 실행되고, cgroup v2를 통해 자원이 격리되는 환경이면 다른 플랫폼에도 적용 가능하다.

플랫폼	호환성	비고
Knative	O	기본 대상
OpenFaaS	O	cgroup v2 사용 시
AWS Lambda	△	Firecracker VM, 제한적 접근
Azure Functions	△	플랫폼 종속적

8.3 NIC Offload 비활성화

필수: NIC의 offload 기능을 비활성화해야 정확한 패킷 단위 측정이 가능하다.

비활성화 이유:

Offload	문제점
TSO (TCP Segmentation Offload)	여러 패킷이 하나로 합쳐져 커널 통과
GSO (Generic Segmentation Offload)	커널에서 보는 패킷 ≠ 실제 wire 패킷
GRO (Generic Receive Offload)	수신 측에서 패킷 합침
LRO (Large Receive Offload)	대형 수신 세그먼트 생성

비활성화 방법:

```
# 인터페이스 확인
ip a

# 현재 설정 확인
ethtool -k <인터페이스>

# Offload 비활성화
ethtool -K <인터페이스> tso off gso off gro off lro off
```

성능 영향: Offload 비활성화는 네트워크 성능에 영향을 줄 수 있다. 특히 대용량 데이터 전송 시 CPU 사용량이 증가한다. 이는 측정 환경의 트레이드오프로 감안해야 한다.

8.4 I/O 스케줄러 설정 (선택)

Block I/O 경합을 명확히 관찰하려면 공정 큐잉 스케줄러 사용을 권장한다.

```
# 현재 스케줄러 확인
cat /sys/block/<device>/queue/scheduler

# BFQ 스케줄러 설정 (공정 큐잉)
echo bfq > /sys/block/<device>/queue/scheduler
```

8.5 권한 요구사항

eBPF 프로그램을 커널에 로드하려면 다음 중 하나가 필요하다:

방법	설명
root 권한	<code>sudo ./puretime</code>
CAP_BPF capability	<code>setcap cap_bpf+ep ./puretime</code> (커널 5.8+)

9. 제약사항 및 한계

9.1 소켓 없는 패킷

네트워크 이벤트에서 `skb->sk`가 NULL인 패킷은 cgroup을 식별할 수 없다.

해당 패킷 유형	처리 방식
Forwarding 패킷	분석에서 제외
일부 시스템 패킷	cgroup_id = 0으로 기록
Raw socket 패킷	제한적 추적

9.2 Softirq 귀속의 한계

Softirq 구간 내에서 처리된 I/O 이벤트를 기반으로 self/other를 분류하지만, 모든 I/O 이벤트가 명확하게 식별되지 않을 수 있다.

특히 문제가 되는 경우:

- 네트워크 수신 경로에서 패킷이 소켓에 연결되기 전 단계
- 여러 cgroup의 I/O가 배치로 처리되는 경우

9.3 타임라인 재구성의 보수성

현재 알고리즘(overlap 제거 방식)은 wait 구간의 overlap만 처리하고, I/O와 APP 간의 병렬성은 고려하지 않는다.

실제 상황:
 [APP 실행][대기][APP 실행]
 [I/O 처리] ← I/O는 별도 하드웨어에서 병렬 처리

현재 알고리즘:
 대기 시간 전체를 wait로 계산 (I/O 처리 시간 중 일부는 실제론 병렬)

이로 인해 noise-free makespan이 **실제보다 약간 클 수 있다**. 단, 서버리스 함수가 직렬 실행되는 특성상 이 차이는 크지 않을 것으로 예상된다.

9.4 측정 오버헤드

eBPF 트레이싱은 커널에 약간의 오버헤드를 추가한다.

오버헤드 원인	영향	완화 방법
Hook 실행	이벤트당 ~100ns	필수 이벤트만 트레이싱
Ring Buffer 처리	메모리 대역폭	256MB 버퍼로 드롭 방지
로그 파일 쓰기	Disk I/O	4KB 버퍼링
Softirq 트레이싱	빈도 높은 이벤트	관련 벡터만 필터링

9.5 메모리/디스크 사용

자원	사용량	고려사항
Ring Buffer	256MB (기본)	조정 가능
로그 파일	가변 (워크로드 의존)	장시간 트레이싱 시 로테이션 필요

10. 관련 연구 및 차별점

10.1 서버리스 성능 분석 연구

연구	접근 방식	PureTime과의 차이
Wang et al. [1]	서버리스 플랫폼의 성능 특성 분석	관찰적 분석 vs 노이즈 제거
Singhvi et al. [2]	저지연 서버리스 플랫폼 설계	플랫폼 설계 vs 측정 도구
Zhang et al. [3]	Harvested 자원에서의 서버리스	자원 수확 vs 노이즈 측정

10.2 성능 간섭 연구

연구	초점	PureTime과의 차이
Kim et al. [4]	서버리스 성능 변동성 분석	현상 분석 vs 노이즈 제거

연구	초점	PureTime과의 차이
Zhao et al. [5]	VM 성능 간섭 서베이	서베이 vs 구체적 솔루션

10.3 eBPF 기반 트레이싱 연구

기존 eBPF 트레이싱 도구(BCC, bpftrace 등)는 범용 트레이싱을 제공하지만, **noisy neighbor 영향을 정량화**하는 데 특화되지 않았다.

10.4 PureTime의 차별화된 기여

1. **노이즈의 직접 측정**: 통계적 추정이 아닌 커널 레벨 직접 측정
2. **인과관계 파악**: "대기 발생" → "다른 테넌트 때문" 연결
3. **Noise-Free Makespan 개념**: 순수 실행 시간의 정량적 정의
4. **컨테이너 수준 귀속**: cgroup 기반의 정확한 이벤트 귀속
5. **실용적 도구 제공**: 연구 프로토타입이 아닌 사용 가능한 시스템

11. 향후 연구 방향

11.1 Network RX 트레이싱

현재는 네트워크 송신(TX) 경로만 트레이싱한다. 수신(RX) 경로의 경우 패킷이 NIC에서 수신되어 애플리케이션까지 전달되는 과정에서 **noisy neighbor 영향**이 있을 수 있다.

도전 과제:

- 수신 경로에서 패킷-cgroup 매핑이 더 복잡함
- TCP 재조립 과정에서의 추적

11.2 LLC Cache Contention

Last Level Cache(LLC) 경합은 현재 트레이싱하지 않는다. LLC 경합은 on-CPU 시간 내에서 발생하여 실행 속도를 늦추지만, 대기 시간으로 나타나지 않아서 현재 방법으로는 측정이 어렵다.

접근 방안:

- perf_event 연동을 통한 하드웨어 카운터 측정
- LLC miss rate와 실행 시간의 상관관계 분석

11.3 이기종 하드웨어 정규화

동일한 함수라도 CPU 클럭, 메모리 속도, 스토리지 종류가 다른 노드에서 실행되면 실행 시간이 달라진다.

확장 방안:

- 노드별 벤치마크 점수 측정
- 실행 시간을 정규화하여 하드웨어 차이까지 제거
- "순수 워크로드 시간" 계산

11.4 실시간 대시보드

현재는 오프라인 분석을 수행하지만, 실시간으로 메트릭을 수집하고 시각화하는 대시보드를 구축할 수 있다.

구현 방안:

- Prometheus exporter를 통한 메트릭 노출
- Grafana 대시보드로 시각화
- 실시간 noisy neighbor 경고 시스템

12. 결론

12.1 핵심 기여 요약

PureTime은 서버리스 컴퓨팅 환경에서 **noisy neighbor**로 인한 성능 노이즈를 정밀하게 측정하고 제거하는 시스템이다.

기술적 기여:

1. eBPF를 활용한 커널 레벨 자원 경합 추적
2. CPU, 네트워크, 블록 I/O, softirq 네 가지 경합 지점에서의 노이즈 측정
3. Interval merge 알고리즘을 통한 정확한 noise-free makespan 계산
4. cgroup 기반 컨테이너 수준 이벤트 귀속

12.2 실용적 의의

PureTime은 다음과 같은 실용적 가치를 제공한다:

활용 시나리오	기존 방식의 문제	PureTime의 해결책
성능 벤치마킹	환경 노이즈로 불공정 비교	노이즈 제거된 순수 성능 비교
카나리 배포	노이즈로 인한 false alarm	실제 성능 회귀만 탐지
리소스 최적화	노이즈 포함된 데이터로 잘못된 결정	정확한 자원 요구량 파악
디버깅	코드 vs 환경 원인 구분 어려움	명확한 원인 분리

12.3 향후 전망

서버리스 컴퓨팅의 채택이 확대됨에 따라, 성능 측정의 신뢰성 문제는 더욱 중요해지고 있다. PureTime이 제시하는 *******노이즈를 직접 측정하고 제거한다*******는 접근 방식은 이 문제에 대한 근본적인 해결책을 제시한다.

향후 Network RX 트레이싱, LLC 캐시 경합 측정, 실시간 대시보드 등의 확장을 통해 더욱 완전한 noise-free 성능 측정 시스템으로 발전할 수 있을 것이다.

성능 평가를 위한 실험

Puretime의 노이즈 제거 정확도 분석

진짜 Noise중에 얼마를 Puretime이 성공적으로 제거했는지 분석

Type	Value
Total Noise (GT)	18(E2E at Contention) - 2(E2E at Solo) = 16s
Removed Noise by Puretime	18(E2E at Contention) - 2.6(E2E Calculated by Puretime) = 15.4s
Remaining Noise	16 - 15.4 = 0.6s
Noise Removal Efficiency	15.4/16 * 100 = 96.25%

Baseline과의 정확도 분석

- 노이즈가 전혀 없는 격리된 환경(Idle state)에서의 실행 시간($T_{isolated}$)과, 의도적으로 노이즈를 주입한 환경에서 PureTime이 계산한 *NoiseFreeMakespan*을 비교

노이즈 유형별 정확도 분석

- CPU Stress, Network Flooding, Disk I/O Heavy 워크로드를 각각 별도로 주입하며 PureTime의 Breakdown 메트릭이 해당 노이즈를 정확히 식별하는지 측정

노이즈 강도별 정확도 분석

- CPU Stress, Network Flooding, Disk I/O Heavy 워크로드를 각각 여러 강도로 주입하며 PureTime의 Breakdown 메트릭이 해당 노이즈를 안정적으로 정확히 식별하는지 측정

시스템 오버헤드 측정

실행 시간 지연 측정

- PureTime을 비활성화했을 때와 활성화했을 때의 함수 1K회 실행 시간을 비교
 - CPU Stress, Network Flooding, Disk I/O Heavy 워크로드에서 (Worst-case) 오버헤드를 측정

자원 소비량 측정

- eBPF Tracer, Loader가 이벤트를 폴링하고 저장하는 과정에서 발생하는 CPU 및 메모리 사용량 측정
 - Puretime 계산은 Backlog로 하기 때문에 Analyzer는 제외

Case Study

False Alarm 잡는 카나리 배포

- 이 실험은 "멀쩡한 코드가 노이즈 때문에 느려졌을 때, 기존 모니터링 툴은 '망했다'고 경보를 올리지만, PureTime은 '이건 노이즈일 뿐 코드는 정상이야'라고 정확히 판단한다"는 것을 보임
 - 또한 Stable한 값이 나오므로, 분산이 줄어들어 카나리 배포의 속도도 빨라짐

per Resource Contention Aware Scheduling

- Knative Pod Autoscaler (KPA)와 비교했을 때, Puretime이 효과적인 것을 보임
 - KPA는 단순히 Concurrency(Pod에서 동시에 잡고있는 요청의 수)를 기준으로 판단
 - $Concurrency \propto RequestsPerSecond \times TimePerRequest$
 - Little's Law로 이렇게 표현할 수 있지만, 실제로는 측정하는 시점에 동시에 잡고있는 것들의 수
 - Concurrency 수치가 목표치를 넘으면 "아, 부하가 늘었구나!"라고 판단하고 Pod를 늘림
 - Noisy neighbor로 인해서 TimePerRequest가 늘어났기 때문에 Concurrency가 늘어난 경우, Puretime은 이에 Scale-out 하지 않고 그대로 유지함
 - Noisy neighbor가 발생하면 리소스가 효율적으로 사용되지 않고 있는 것
 - Noisy neighbor를 종료하거나, Migration 하는 등 대처를 할 수 있게 스케줄러에 시그널 제공
 - 잘못된 Concurrency 증가 탐지를 기반으로 그대로 함수 인스턴스(Pod)를 늘렸다면, 불필요하게 함수 인스턴스를 띄움으로 인해 Over-provisioning 발생